

WHATSNEXT VISION MOTORS

Salesforce CRM Implementation for Vehicle Sales and Customer Management

CHAPTER 1 – REQUIREMENT ANALYSIS AND PLANNING

1.1 Introduction

Every Salesforce implementation begins with a clear understanding of the business it is meant to serve. This chapter documents the requirement-gathering and planning activities carried out for WhatsNext Vision Motors, a fictitious vehicle sales and service organization that operates through multiple dealers. Before a single object or field was created in the Salesforce org, the underlying business process – how a customer discovers a vehicle, books a test drive, places an order, and later requests service – was studied and translated into a set of functional and technical requirements. This chapter presents that analysis and the resulting project plan that guided the remaining four phases of development.

1.2 Understanding Business Requirements

WhatsNext Vision Motors sells vehicles through a network of regional dealers. Prior to this project, dealer stock, customer enquiries, test-drive bookings, and service requests were tracked using disconnected spreadsheets and phone calls. This created three recurring problems that the CRM was expected to solve.

User Needs

- Sales staff need a single place to see vehicle stock, dealer availability, and order status.
- Customers need a fast and reliable way to book a test drive and receive a reminder before the appointment.
- Dealers need vehicle orders to be routed to them automatically instead of manually re-keyed from spreadsheets.
- Service advisors need a structured way to log and track vehicle service requests.
- Management needs reports and dashboards summarising orders, stock levels, and service activity.

Existing Business Problems

- No centralised system to track vehicle stock across dealers, resulting in orders being accepted for vehicles that were already out of stock.
- Dealer assignment for a new order was done manually, which delayed order processing and caused inconsistent workloads across dealers.
- Customers frequently missed test drives because no reminder was sent ahead of the scheduled appointment.
- Orders placed when stock was unavailable had to be manually tracked and re-confirmed once new stock arrived, which was easy to forget.

- There was no reporting visibility into how many vehicles were sold, ordered, or awaiting service at any point in time.

Why Salesforce CRM is Required

Salesforce Lightning Platform was selected because it allows both declarative configuration (objects, fields, validation rules, flows) and programmatic development (Apex triggers, batch jobs, scheduled jobs) to be combined within a single, secure, cloud-hosted platform. This meant that WhatsNext Vision Motors could automate dealer assignment and stock validation using native Salesforce automation tools, without the cost or complexity of building and hosting a custom application from scratch, while still retaining the flexibility to implement complex business rules using Apex where declarative tools were not sufficient.

1.3 Problem Statement

WhatsNext Vision Motors requires a Salesforce CRM application that can store vehicle inventory and dealer information, capture customer and order details, automatically assign the nearest available dealer to every new order, prevent orders from being placed against vehicles with zero stock, remind customers of upcoming test drives, and automatically confirm orders that were pending due to stock shortage once new stock is added – all while giving management reporting visibility into the entire sales and service pipeline.

1.4 Proposed Solution

The proposed solution is a Salesforce Lightning application, branded as “WhatsNext Vision Motors,” built on six custom objects that model vehicles, dealers, customers, orders, test drives, and service requests. Record-Triggered Flows handle dealer auto-assignment and test-drive reminder emails declaratively, while a custom Apex trigger and trigger handler class enforce stock validation at the moment an order is created or updated. A Batch Apex class, invoked by a Scheduled Apex class running once every day, automatically re-processes orders that were left pending due to insufficient stock and confirms them once stock has been replenished. Lightning App Builder, custom tabs, and page layouts deliver a focused user experience, and Salesforce Reports and Dashboards give management real-time visibility into orders, stock, and service activity.

1.5 Project Scope

- In scope: Vehicle inventory management, dealer management, customer management, order management, test-drive scheduling, service request tracking, dealer auto-assignment automation, stock validation, test-drive reminder emails, batch-driven pending order confirmation, reports and dashboards.
- Out of scope (identified as future enhancements): payment processing/invoicing integration, Experience Cloud customer self-service portal, native mobile application, AI-based recommendations, and real-time GPS-based dealer mapping.

1.6 Project Objectives

- To design a normalised Salesforce data model that accurately represents vehicles, dealers, customers, orders, test drives, and service requests.
- To automate dealer assignment for every new vehicle order using a Record-Triggered Flow.
- To prevent vehicle orders from being created or updated against out-of-stock vehicles using Apex-based validation.
- To send automated reminder emails to customers ahead of a scheduled test drive.
- To build a Batch Apex and Scheduled Apex mechanism that automatically confirms pending orders once stock is replenished.
- To provide a branded, role-appropriate Lightning user interface with reports and dashboards for management visibility.
- To document, test, and deploy the application in line with SmartBridge evaluation guidelines.

1.7 Business Value Mapping

The table below maps each project objective to the tangible business value it delivers to WhatsNext Vision Motors.

Objective	Business Value
Centralised vehicle, dealer and customer data model	Single source of truth eliminates duplicate and conflicting spreadsheets across dealers
Automated dealer assignment (Record-Triggered Flow)	Reduces order processing time and removes manual routing errors
Stock validation on Vehicle Order (Apex trigger)	Prevents overselling and protects customer trust
Automated test-drive reminder emails	Reduces missed appointments and improves customer experience
Batch Apex + Scheduled Apex for pending order confirmation	Ensures no order is forgotten once stock is replenished, without manual follow-up
Reports and Dashboards	Gives management real-time visibility into sales, stock and service performance

1.8 Functional Requirements

ID	Requirement
FR-01	The system shall allow authorised users to create, view, edit and delete Vehicle records with stock quantity and price.
FR-02	The system shall allow authorised users to create and maintain Vehicle Dealer records.
FR-03	The system shall allow authorised users to create and maintain Vehicle Customer records.

ID	Requirement
FR-04	The system shall allow a Vehicle Order to be created against a specific Vehicle and Vehicle Customer.
FR-05	The system shall automatically assign the nearest available dealer to a new Vehicle Order.
FR-06	The system shall prevent a Vehicle Order from being saved if the related Vehicle has zero stock.
FR-07	The system shall allow a Vehicle Test Drive to be scheduled for a Vehicle Customer against a specific Vehicle.
FR-08	The system shall send a reminder email to the customer ahead of a scheduled test drive.
FR-09	The system shall allow a Vehicle Service Request to be logged against a Vehicle and Vehicle Customer.
FR-10	The system shall automatically re-evaluate and confirm pending Vehicle Orders once stock is replenished, on a scheduled basis.
FR-11	The system shall provide reports and dashboards summarising orders, stock and service requests.

1.9 Non-Functional Requirements

ID	Requirement	Description
NFR-01	Usability	The Lightning interface must be intuitive enough for dealer staff with minimal Salesforce experience.
NFR-02	Performance	Automated flows and Apex logic must execute within Salesforce governor limits without noticeable delay to the user.
NFR-03	Security	Access to records must be controlled through Profiles, Permission Sets, and Field-Level Security.
NFR-04	Maintainability	Apex classes must follow a trigger-handler pattern to keep business logic testable and easy to extend.
NFR-05	Scalability	Batch Apex must be used for bulk processing of pending orders so the design scales as order volume grows.
NFR-06	Reliability	Scheduled Apex jobs must run daily without manual intervention and be verifiable through the Scheduled Jobs page.

1.10 Data Model Design

Six custom objects form the core data model of the application, connected through lookup relationships so that a Vehicle Order always traces back to a specific Vehicle and Vehicle Customer, and a Vehicle can be linked to the Vehicle Dealer currently holding its stock. The relationships are summarised below and explained in full, object by object, in Chapter 2.

Custom Object	Purpose	Key Relationships
Vehicle	Stores each vehicle model, its stock quantity, price and status	Lookup to Vehicle Dealer
Vehicle Dealer	Stores dealer name, location, phone and email	Parent of Vehicle (via Dealer lookup)
Vehicle Customer	Stores customer contact details and preferred vehicle type	Parent of Vehicle Order, Vehicle Test Drive, Vehicle Service Request
Vehicle Order	Represents a customer's order for a specific vehicle	Lookup to Vehicle, Lookup to Vehicle Customer
Vehicle Test Drive	Represents a scheduled test drive for a customer and vehicle	Lookup to Vehicle, Lookup to Vehicle Customer
Vehicle Service Request	Represents a service request logged for a vehicle	Lookup to Vehicle, Lookup to Vehicle Customer

1.11 Security Model

Access to WhatsNext Vision Motors data is controlled through the standard Salesforce security model. During development, the System Administrator profile was used to create and configure all objects, fields, flows, and Apex classes. For a production rollout, the security model is designed as follows:

- **Profiles:** A System Administrator profile retains full configuration access; a Dealer Staff profile (planned for future rollout) would be granted read/write access to Vehicle, Vehicle Order and Vehicle Test Drive records only.
- **Permission Sets:** Additional, granular permissions (for example, the ability to close a Vehicle Service Request) would be layered on top of base profiles using Permission Sets rather than by creating new profiles for every variation.
- **Sharing Rules:** Default organisation-wide sharing is Private for sensitive objects such as Vehicle Order and Vehicle Customer, with sharing rules extending visibility to a dealer's own orders once dealer-based roles are introduced.
- **Field-Level Security:** Sensitive fields (for example, customer Email and Phone) are restricted to profiles that need them for day-to-day work.

CHAPTER 2 – SALESFORCE DEVELOPMENT (BACKEND & CONFIGURATION)

This chapter is the technical core of the project. It documents every backend configuration carried out in the Salesforce Developer Edition organization to build WhatsNext Vision Motors – from the initial environment setup through custom objects, fields, validation rules, the Lightning app, tabs, page layouts, and the two Record-Triggered Flows that automate dealer assignment and test-drive reminders.

2.1 Salesforce Developer Environment Setup

A free Salesforce Developer Edition organization was provisioned for this project. Developer Edition provides a fully-featured sandbox of the Salesforce Lightning Platform – including Lightning App Builder, Flow Builder, Apex, and reporting tools – at no cost, which makes it the ideal environment for a mini-project of this scope. All configuration, coding, and testing described in this report was carried out inside this org.

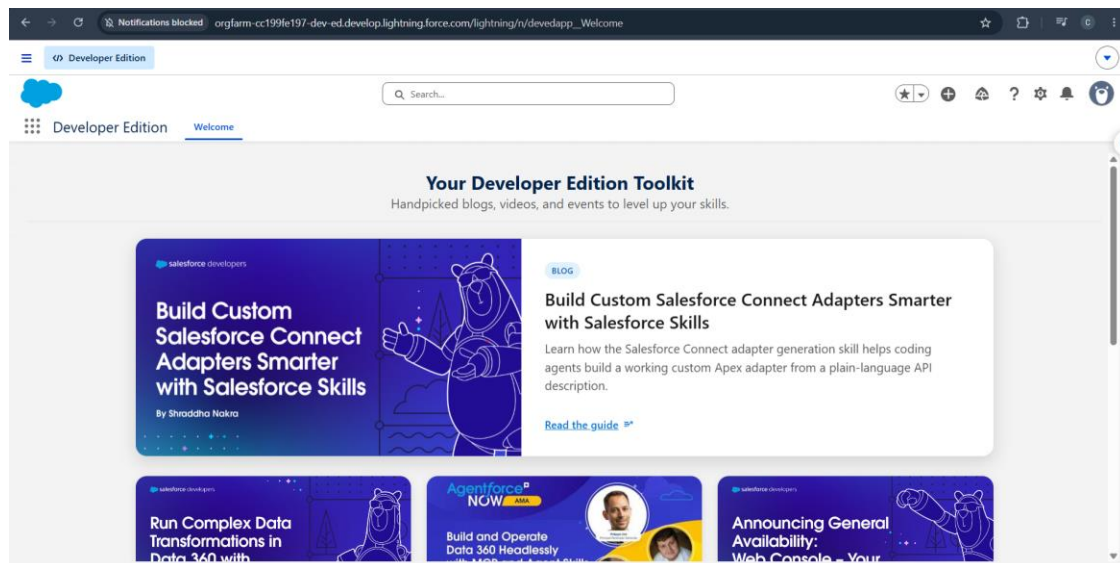


Figure 2.1: Salesforce Developer Edition home page after successful org login.

2.2 Custom Objects Creation

Six custom objects were created through Setup → Object Manager to model the WhatsNext Vision Motors business process end to end. Each object is described below along with its purpose, singular/plural labels, and business usage.

2.2.1 Vehicle

Purpose: Stores every vehicle model available for sale, including its stock quantity, price, and current status. Record Name: Vehicle Name (Text). Business usage: The Vehicle object is the anchor for stock validation – every Vehicle Order looks up to a Vehicle record, and the Apex trigger described in Chapter 4 checks this record's stock quantity before allowing an order to be saved.

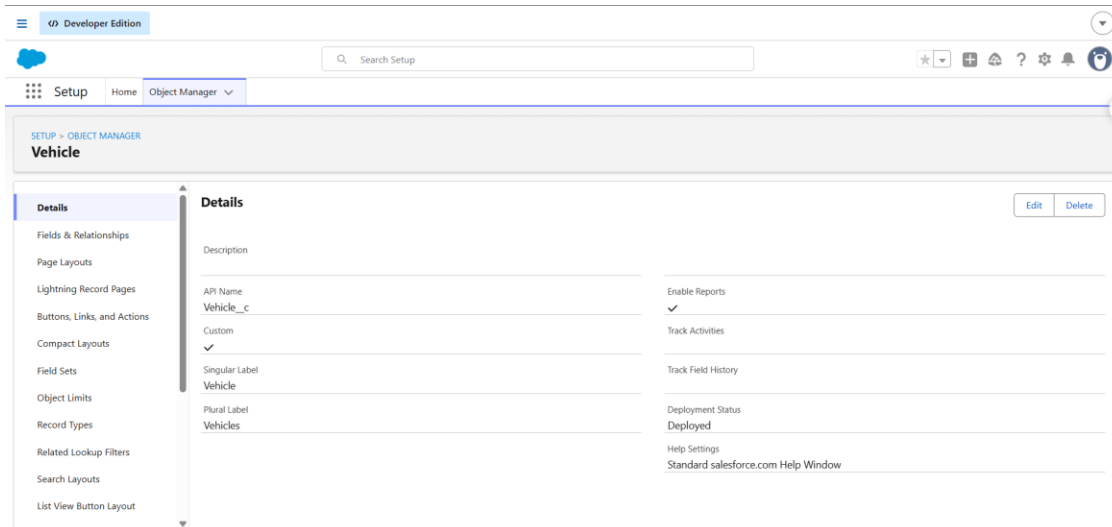


Figure 2.2: Object Manager detail page for the Vehicle custom object (API name Vehicle__c).

2.2.2 Vehicle Dealer

Purpose: Stores dealer information – name, location, phone, and email – for every dealer in the WhatsNext Vision Motors network. Record Name: Vehicle Dealer Name (Text). Business usage: Vehicle records look up to Vehicle Dealer, and the Auto Assign Dealer flow (Section 2.8) uses this object to find the nearest available dealer for a new order.

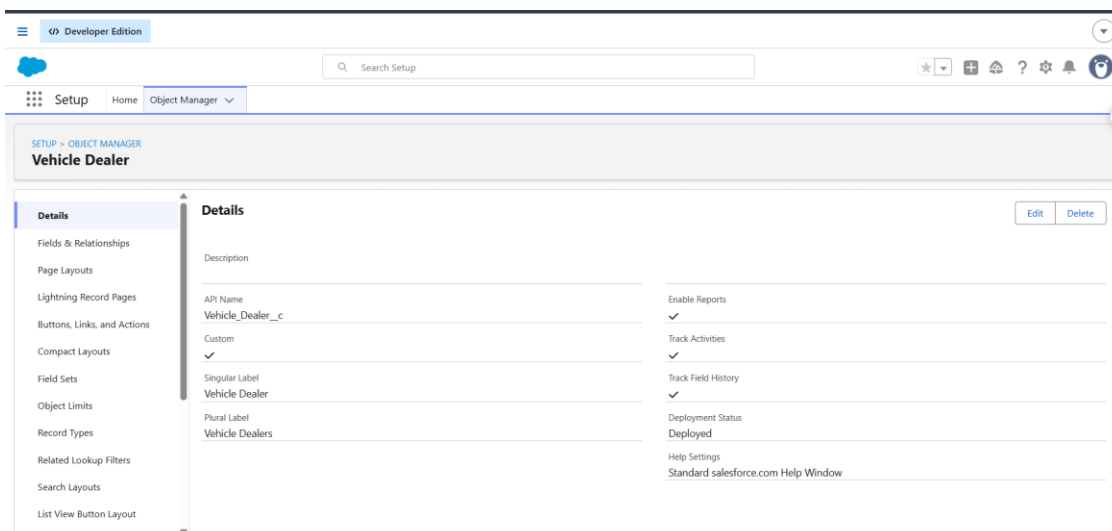


Figure 2.3: Object Manager detail page for the Vehicle Dealer custom object (API name Vehicle_Dealer__c).

2.2.3 Vehicle Customer

Purpose: Stores customer contact information and preferred vehicle type. Record Name: Vehicle Customer Name (Text). Business usage: Every Vehicle Order, Vehicle Test Drive, and Vehicle Service Request looks up to a Vehicle Customer record, making it the central point of contact information used by the reminder-email flow.

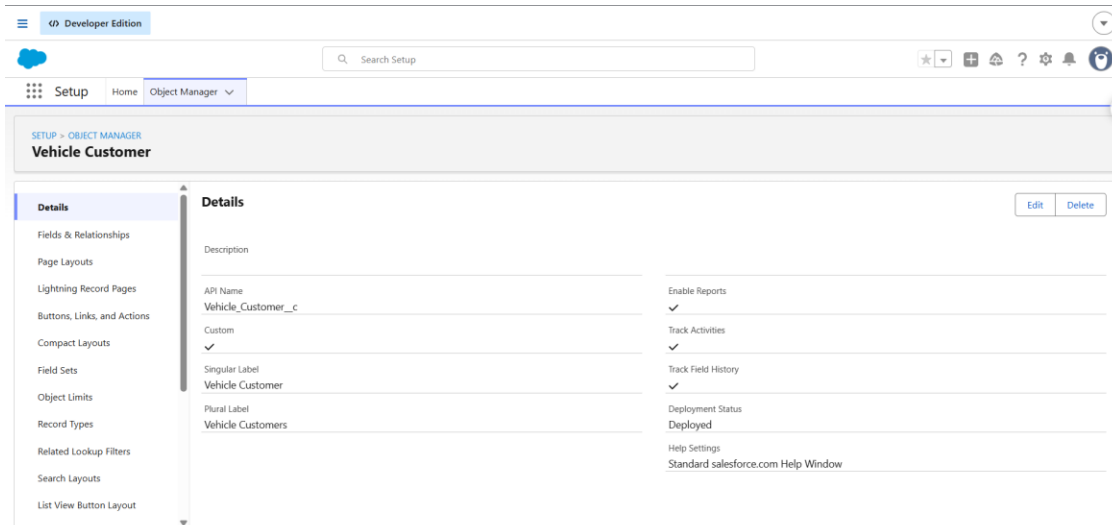


Figure 2.4: Object Manager detail page for the Vehicle Customer custom object (API name Vehicle_Customer__c).

2.2.4 Vehicle Order

Purpose: Represents a customer's order for a specific vehicle and tracks its status through the order lifecycle. **Record Name:** Vehicle Order Name (Text). **Business usage:** This is the object on which the stock-validation Apex trigger and dealer auto-assignment flow both fire, making it the operational heart of the application.

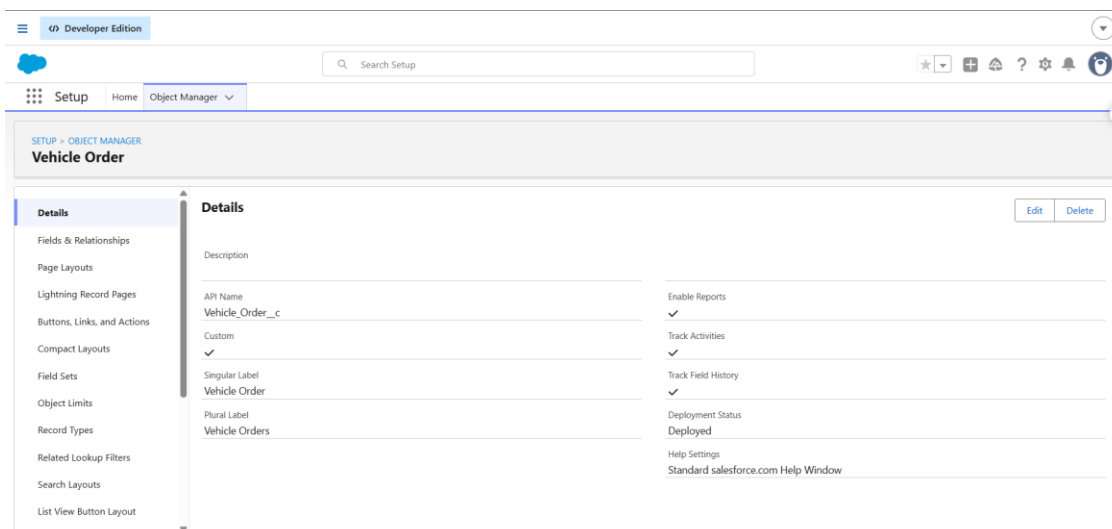


Figure 2.5: Object Manager detail page for the Vehicle Order custom object (API name Vehicle_Order__c).

2.2.5 Vehicle Test Drive

Purpose: Represents a test drive scheduled by a customer for a specific vehicle, including the scheduled date and status. **Record Name:** Vehicle Test Drive Name (Text). **Business usage:** This object drives the Test Drive Reminder flow (Section 2.9), which sends an email to the customer ahead of the scheduled date.

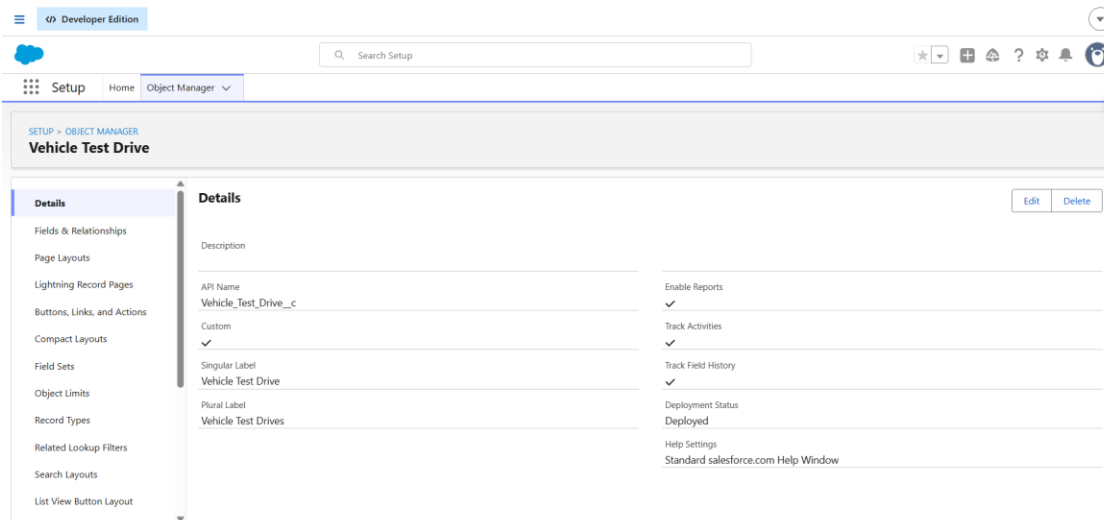


Figure 2.6: Object Manager detail page for the Vehicle Test Drive custom object (API name Vehicle_Test_Drive__c).

2.2.6 Vehicle Service Request

Purpose: Represents a post-sale service request logged against a vehicle owned by a customer, including an issue description, service date, and status. **Record Name:** Vehicle Service Request Name (Text). **Business usage:** Gives the service team a structured place to log and track service issues instead of relying on phone calls and email threads.

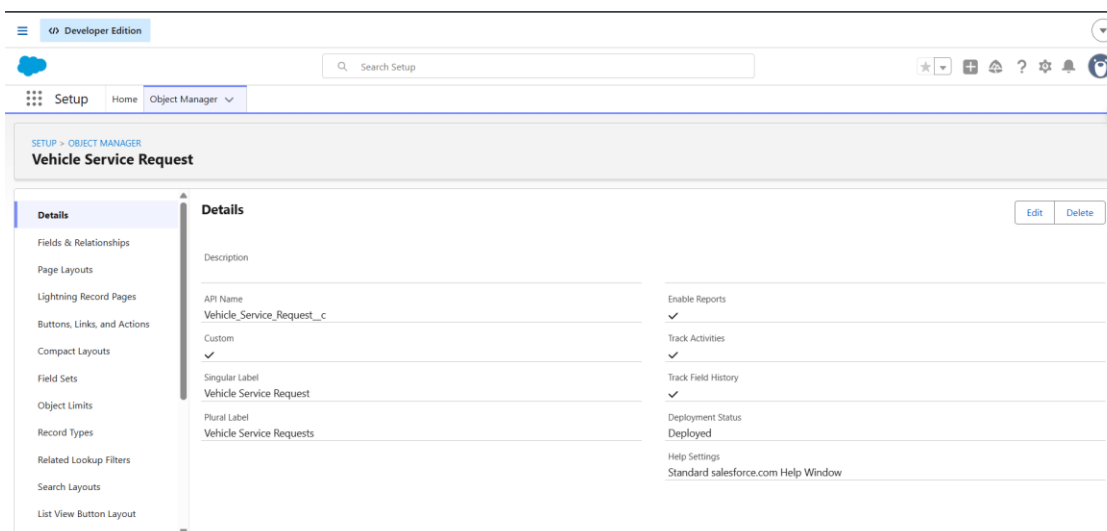


Figure 2.7: Object Manager detail page for the Vehicle Service Request custom object (API name Vehicle_Service_Request__c).

2.3 Fields & Relationships

Each custom object was extended with a set of fields covering picklists, lookup relationships, number fields, email fields, and date fields, as required by the business process. Lookup relationships (rather than master-

detail) were used throughout, since child records such as Vehicle Order should be able to exist and be reassigned independently of a single parent record being deleted.

2.3.1 Vehicle – Fields & Relationships

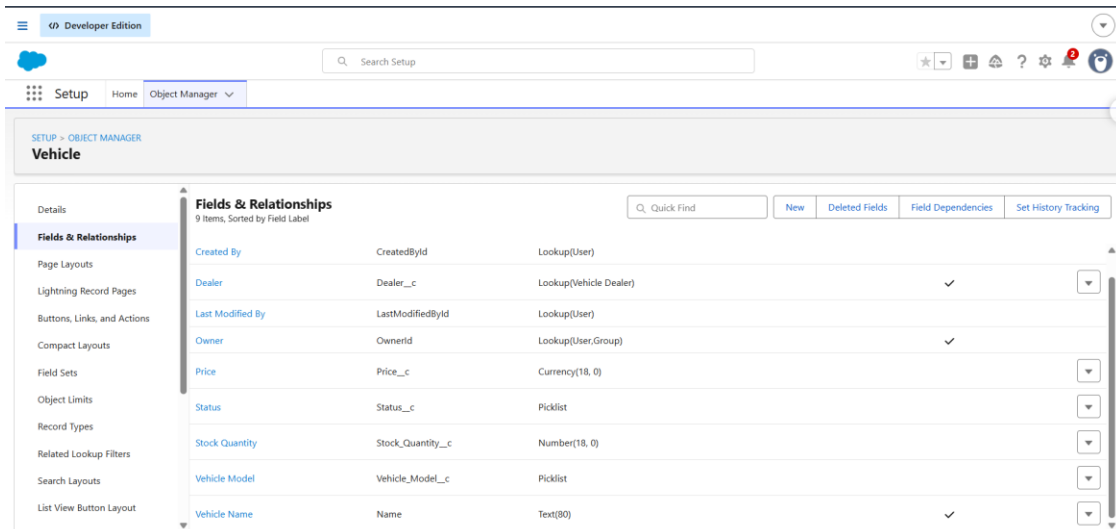


Figure 2.8: Fields & Relationships list for the Vehicle object.

Field Name	Data Type	Purpose
Vehicle Name	Text (80)	Primary record name / model identifier
Dealer	Lookup (Vehicle Dealer)	Links the vehicle to the dealer currently holding its stock
Price	Currency (18,0)	Selling price of the vehicle
Status	Picklist	Current status of the vehicle (e.g., Available, Out of Stock)
Stock Quantity	Number (18,0)	Number of units available; checked by the stock-validation trigger
Vehicle Model	Picklist	Model / variant of the vehicle

2.3.3 Vehicle Customer – Fields & Relationships

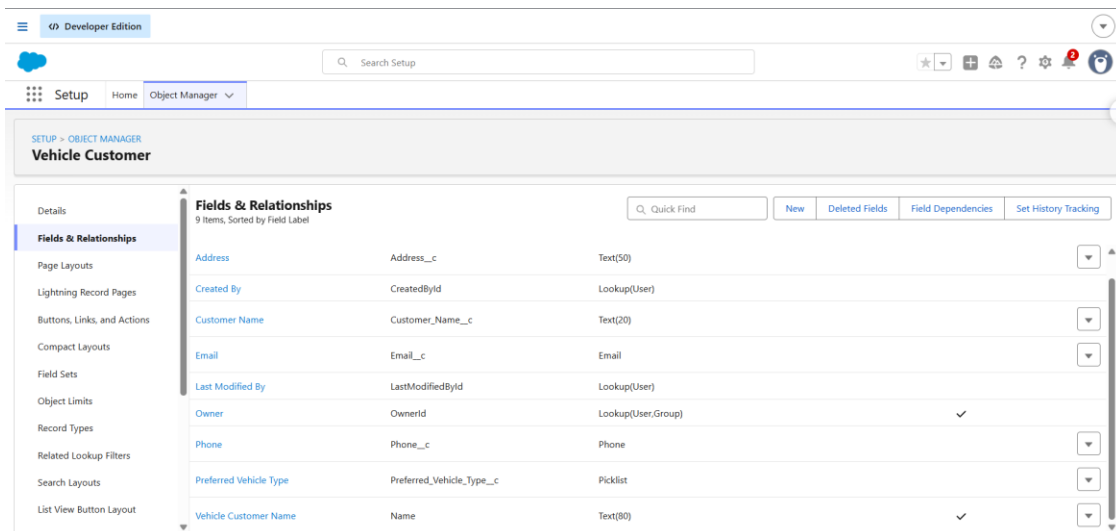
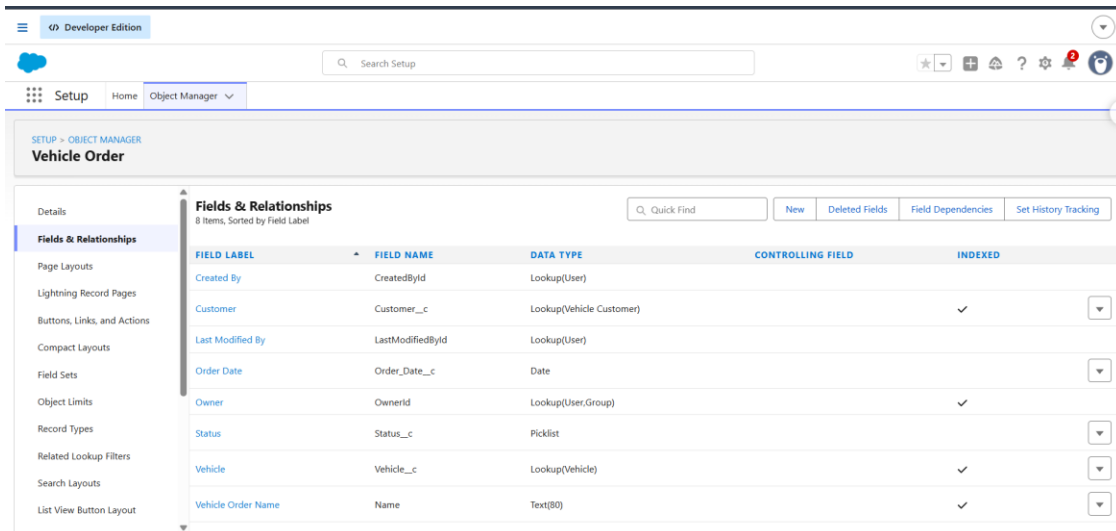


Figure 2.10: Fields & Relationships list for the Vehicle Customer object.

Field Name	Data Type	Purpose
Vehicle Customer Name	Text (80)	Primary record name
Customer Name	Text (20)	Display name of the customer
Address	Text (50)	Customer's mailing address
Email	Email	Used by the test-drive reminder email action
Phone	Phone	Customer contact phone number
Preferred Vehicle Type	Picklist	Customer's stated vehicle preference, used for sales follow-up

2.3.4 Vehicle Order – Fields & Relationships



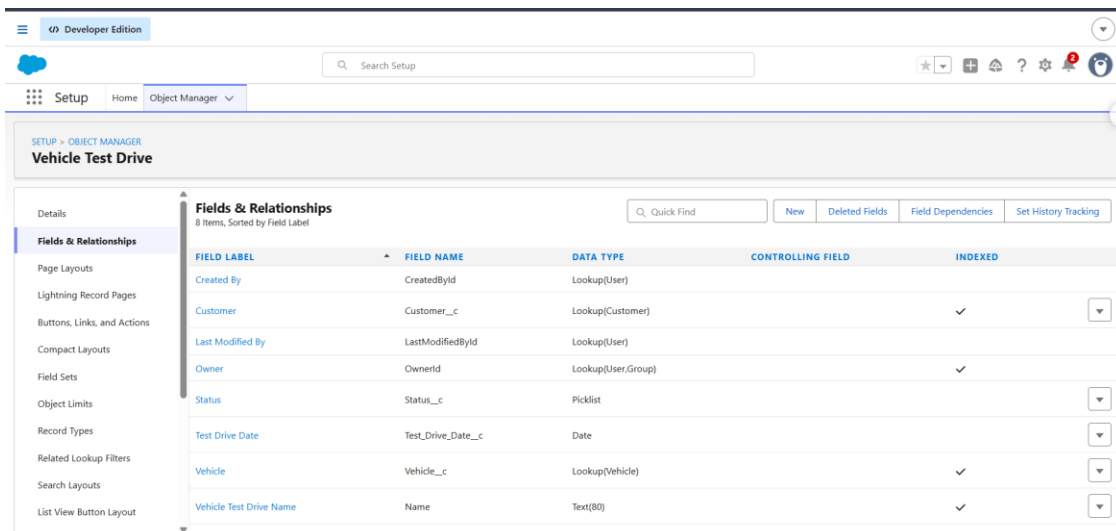
The screenshot shows the Salesforce Setup interface in Developer Edition. The 'Object Manager' tab is selected, and the 'Vehicle Order' object is chosen. The 'Fields & Relationships' section is active, displaying a list of 8 fields. The fields are sorted by Field Label. The table includes columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed status. The fields listed are: Created By (Lookup(User)), Customer (Lookup(Vehicle Customer)), Last Modified By (Lookup(User)), Order Date (Date), Owner (Lookup(User,Group)), Status (Picklist), Vehicle (Lookup(Vehicle)), and Vehicle Order Name (Text(80)).

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Customer	Customer__c	Lookup(Vehicle Customer)		✓
Last Modified By	LastModifiedById	Lookup(User)		
Order Date	Order_Date__c	Date		
Owner	OwnerId	Lookup(User,Group)		✓
Status	Status__c	Picklist		
Vehicle	Vehicle__c	Lookup(Vehicle)		✓
Vehicle Order Name	Name	Text(80)		✓

Figure 2.11: Fields & Relationships list for the Vehicle Order object.

Field Name	Data Type	Purpose
Vehicle Order Name	Text (80)	Primary record name
Customer	Lookup (Vehicle Customer)	The customer placing the order
Vehicle	Lookup (Vehicle)	The vehicle being ordered; used for stock validation
Order Date	Date	Date the order was placed
Status	Picklist	Order lifecycle status (e.g., Pending, Confirmed)

2.3.5 Vehicle Test Drive – Fields & Relationships



The screenshot shows the Salesforce Setup interface in Developer Edition. The 'Object Manager' tab is selected, and the 'Vehicle Test Drive' object is chosen. The 'Fields & Relationships' section is active, displaying a list of 8 fields. The fields are sorted by Field Label. The table includes columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed status. The fields listed are: Created By (Lookup(User)), Customer (Lookup(Customer)), Last Modified By (Lookup(User)), Owner (Lookup(User,Group)), Status (Picklist), Test Drive Date (Date), Vehicle (Lookup(Vehicle)), and Vehicle Test Drive Name (Text(80)).

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Customer	Customer__c	Lookup(Customer)		✓
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Status	Status__c	Picklist		
Test Drive Date	Test_Drive_Date__c	Date		
Vehicle	Vehicle__c	Lookup(Vehicle)		✓
Vehicle Test Drive Name	Name	Text(80)		✓

Figure 2.12: Fields & Relationships list for the Vehicle Test Drive object.

Field Name	Data Type	Purpose
Vehicle Test Drive Name	Text (80)	Primary record name
Customer	Lookup (Vehicle Customer)	Customer scheduled for the test drive
Vehicle	Lookup (Vehicle)	Vehicle to be test-driven
Test Drive Date	Date	Scheduled date of the test drive
Status	Picklist	Test drive status (e.g., Scheduled, Completed)

2.3.6 Vehicle Service Request – Fields & Relationships

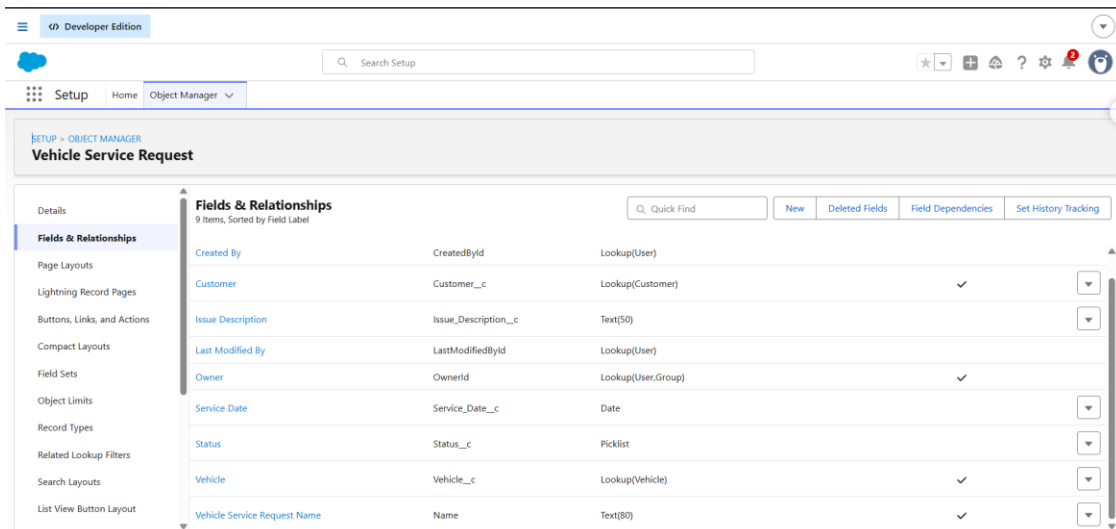


Figure 2.13: Fields & Relationships list for the Vehicle Service Request object.

Field Name	Data Type	Purpose
Vehicle Service Request Name	Text (80)	Primary record name
Customer	Lookup (Vehicle Customer)	Customer who raised the service request
Vehicle	Lookup (Vehicle)	Vehicle the service request relates to
Issue Description	Text (50)	Short description of the reported issue
Service Date	Date	Date the service is requested/scheduled
Status	Picklist	Service request status (e.g., Open, Closed)

2.3.7 Field-Level Detail Examples

To illustrate how individual fields were configured, the detail definitions of two representative fields on the Vehicle object are shown below: the numeric stock field used by the validation logic, and the Dealer lookup relationship.

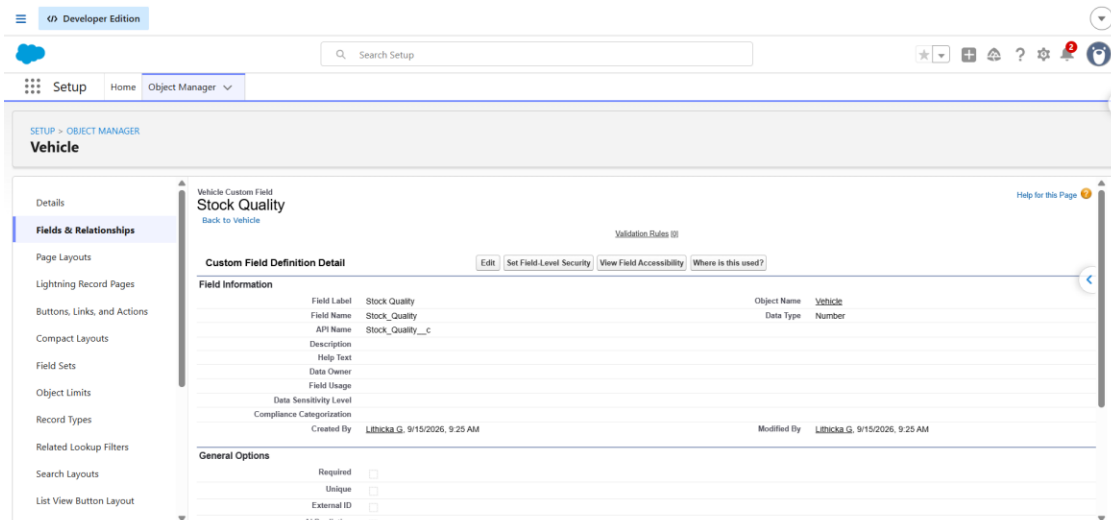


Figure 2.14: Custom field definition detail for the Vehicle stock field (Number data type), used by the stock-validation Apex trigger.

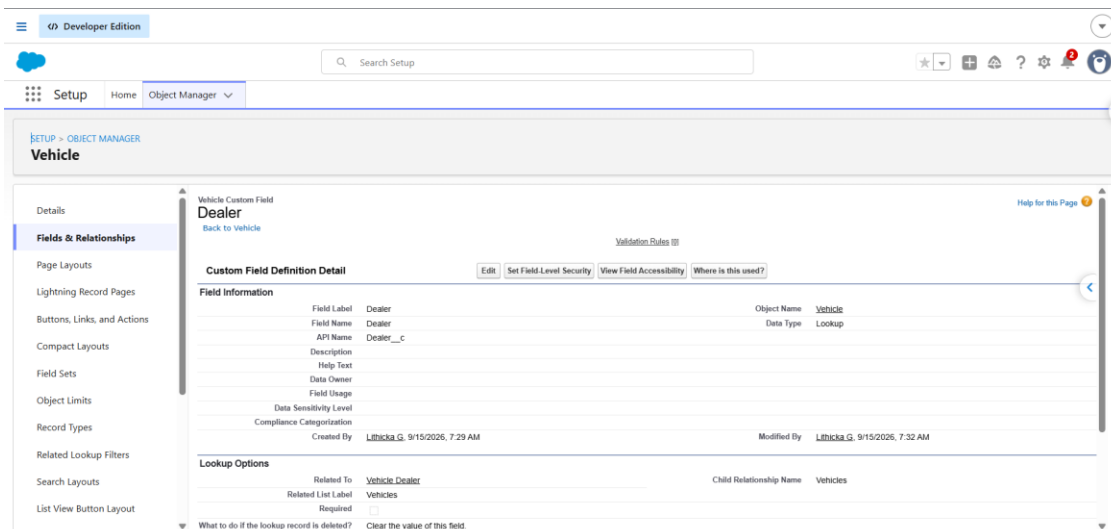


Figure 2.15: Custom field definition detail for the Dealer lookup field on the Vehicle object, showing the related list label 'Vehicles' on Vehicle Dealer.

2.4 Validation Rules

Objective

To reinforce the Apex-based stock check (Chapter 4) with a simple, declarative safeguard, the business rule that a Vehicle Order should not reference a Vehicle whose stock has fallen to zero was also considered at the validation-rule level during design, so that data-quality checks are enforced as close to the data as possible.

Logic

The rule design checks the Stock Quantity field on the related Vehicle at the time a Vehicle Order is created or edited, and blocks the save operation with a descriptive error message if the value is less than or equal to zero, directing the user to select a different vehicle or wait for restock.

Business Impact

This prevents dealers from accidentally promising a vehicle that is not physically available, protecting customer trust and avoiding order cancellations after the fact. In this implementation, the equivalent and more flexible enforcement of this rule was ultimately carried out in Apex (Section 4.1) so that the same logic could also support bulk updates safely; the validation-rule approach documented here represents the declarative alternative evaluated during design.

2.5 Lightning App Configuration

A dedicated Lightning app named “WhatsNext Vision Motors” was created through App Manager so that all six custom objects are grouped under one branded navigation bar rather than being scattered across the generic Salesforce setup menu. The App Manager was used to define the app’s navigation items (Vehicle Customers, Vehicles, Vehicle Dealers, Vehicle Orders, Vehicle Service Requests, Vehicle Test Drives), assign the System Administrator profile to the app during development, and apply the Salesforce standard branding theme.

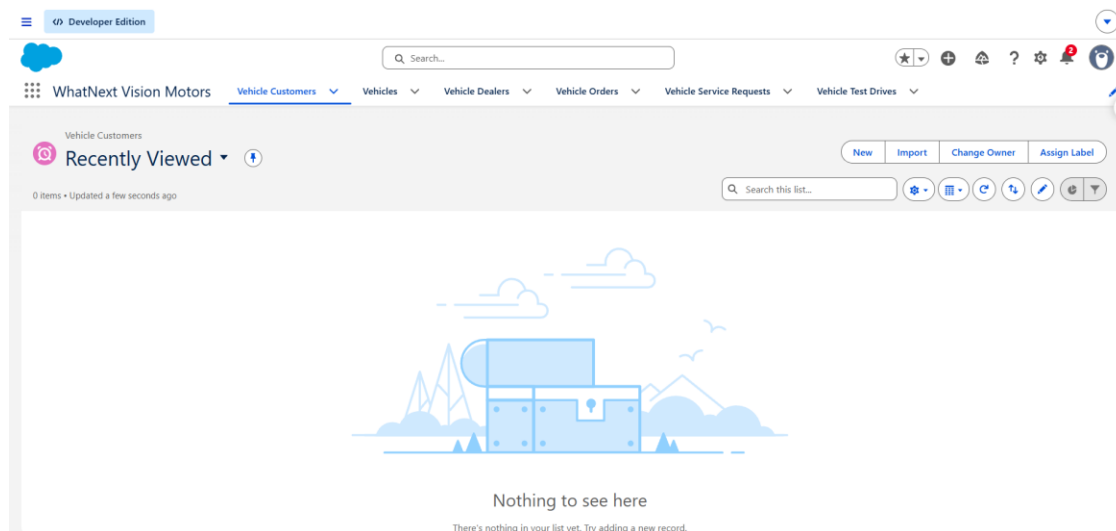


Figure 2.17: WhatsNext Vision Motors Lightning app, showing the custom navigation bar with all six object tabs.

2.6 Lightning Tabs

A custom tab was created for each of the six custom objects using Setup → Tabs → Custom Object Tabs, each with a distinct tab style/icon so that users can visually distinguish between Vehicles, Vehicle Dealers, Vehicle Customers, Vehicle Orders, Vehicle Service Requests, and Vehicle Test Drives at a glance. These tabs were then added to the WhatsNext Vision Motors app’s navigation bar shown in Figure 2.17.

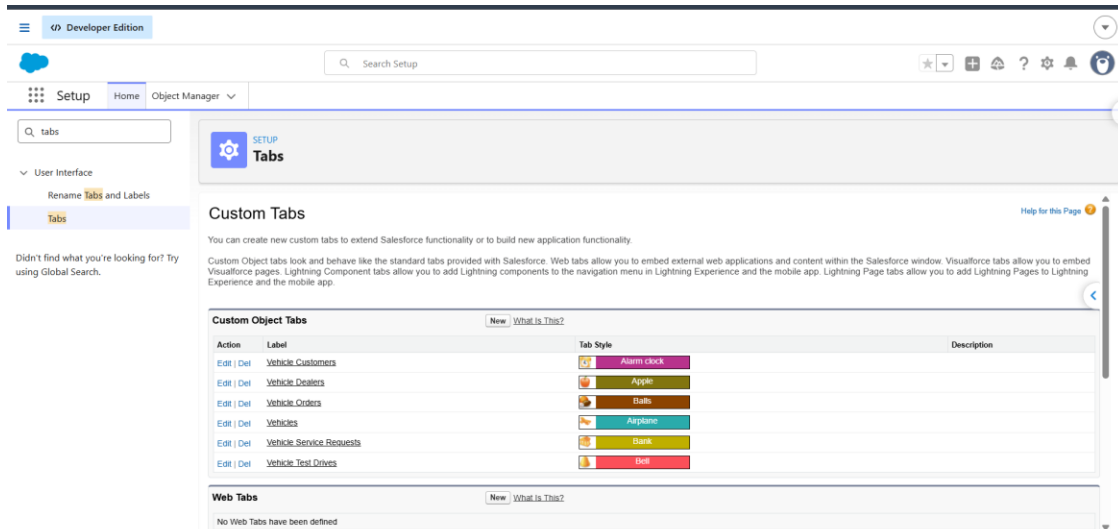


Figure 2.18: Custom Object Tabs configuration page listing all six tabs created for WhatsNext Vision Motors.

2.7 Page Layout Configuration

The default page layout generated for each object was customised to group related fields logically – for example, placing Vehicle, Customer and Status together in the primary section of the Vehicle Order layout – and to surface the correct related lists (such as Vehicle Orders and Vehicle Test Drives on the Vehicle page layout) so that users can navigate between related records without leaving the record page.

2.8 Record-Triggered Flow – Auto Assign Dealer

Objective

To automatically assign the most appropriate Vehicle Dealer to a Vehicle Order as soon as it is created, removing the need for a sales user to manually look up and select a dealer.

Trigger Configuration

The flow is configured as a Record-Triggered Flow on the Vehicle Order object, set to run when a record is created, with the trigger optimised for “Actions and Related Records” so that the Get Records and Update Records elements execute safely within the same transaction.

Flow Logic

- Start: Record-Triggered Flow fires immediately when a new Vehicle Order record is created.
- Get Customer Information: retrieves the related Vehicle Customer record so that customer details are available to the flow.
- Get Nearest Dealer: queries Vehicle Dealer records to identify the dealer that should service this order.
- Assign Dealer to Order: updates the Vehicle Order record, setting the Dealer reference so the order is now routed to a specific dealer.

- End: the flow terminates once the update is complete.

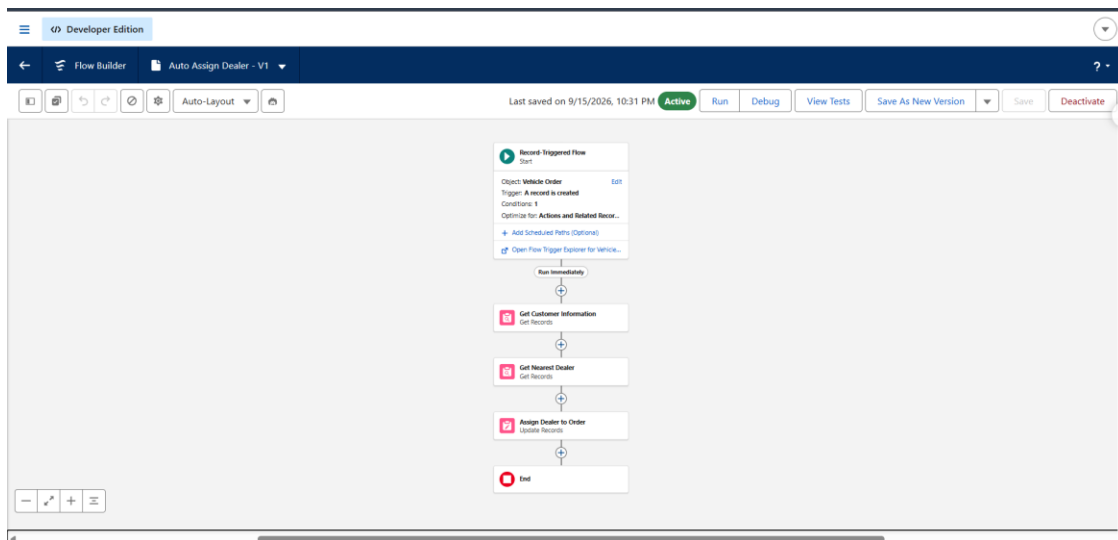


Figure 2.20: Flow Builder canvas for the 'Auto Assign Dealer' Record-Triggered Flow, showing Get Records, Get Nearest Dealer, and Update Records elements in sequence.

Testing Outcome

The flow was activated (status Active, as shown in Figure 2.20) and verified by creating new Vehicle Order records through the Lightning app; in each case the Dealer field on the order was populated automatically without any manual intervention, confirming the automation worked as designed.

2.9 Record-Triggered Flow – Test Drive Reminder

Scheduled Path

This flow is configured on the Vehicle Test Drive object with two paths from a single Record-Triggered start element: an immediate path that retrieves customer information right away, and a Scheduled Path (labelled “Reminder Before Test Drive”) that fires at a defined time offset before the Test Drive Date field, ensuring the reminder email is sent ahead of the appointment rather than at the moment of record creation.

Get Customer → Send Email Action

On the immediate path, a Get Records element (Get Customer Information) retrieves the related Vehicle Customer record so that the customer’s email address is available, and a Send Email action (Send Test Drive Reminder) then dispatches a reminder email using that address.

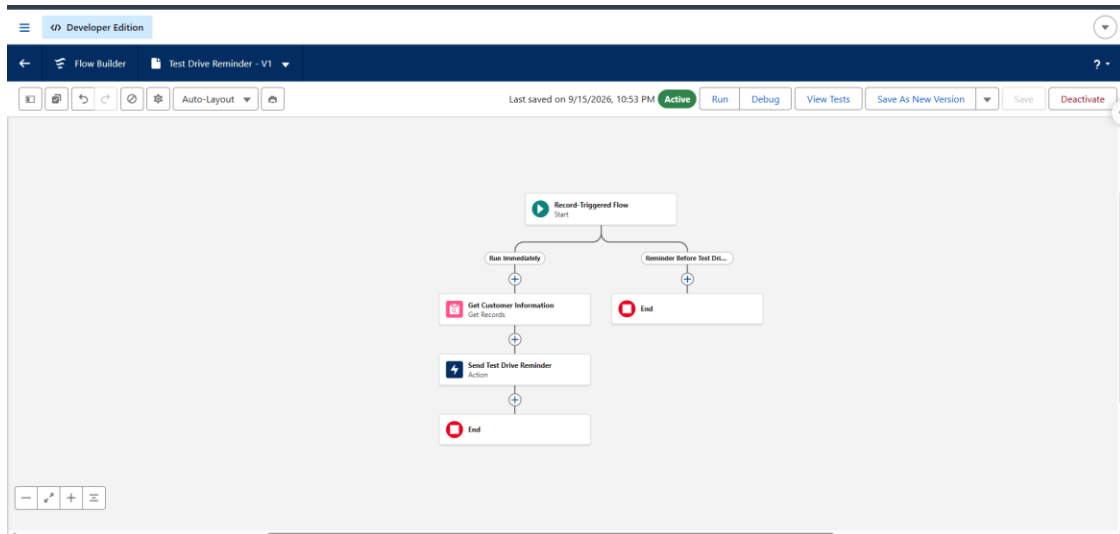


Figure 2.21: Flow Builder canvas for the 'Test Drive Reminder' Record-Triggered Flow, showing the immediate path (Get Customer Information → Send Test Drive Reminder) and the Scheduled Path branch.

Testing Explanation

The flow was activated and validated by creating and updating Vehicle Test Drive records with a future Test Drive Date; the Send Email action executed successfully on the immediate path during testing, confirming that customer information was correctly retrieved and that the email action was wired to the correct recipient field.

CHAPTER 3 – UI/UX DEVELOPMENT AND CUSTOMIZATION

With the data model, automation, and app framework in place from Chapter 2, this chapter focuses on shaping how users actually experience WhatsNext Vision Motors – the layout of the Lightning app, navigation, forms, reports, dashboards, and user access.

3.1 Lightning App Builder

Lightning Experience is Salesforce’s modern, component-based user interface, and Lightning App Builder is the point-and-click tool used to design record pages, home pages, and app pages from a palette of standard and custom Lightning components. For WhatsNext Vision Motors, Lightning App Builder was used to arrange the Highlights Panel, Related Lists, and key fields on each object’s record page so that the most business-critical information (for example, Stock Quantity on a Vehicle, or Status on a Vehicle Order) is visible immediately without scrolling.

3.2 Navigation Bar Customization

The navigation bar of the WhatsNext Vision Motors app (see Figure 2.17 in Chapter 2) was customised to present the six custom tabs in the order that mirrors the real-world sales process – Vehicle Customers, Vehicles, Vehicle Dealers, Vehicle Orders, Vehicle Service Requests, and Vehicle Test Drives – so that a user moving through a typical workflow encounters tabs in a logical, left-to-right sequence.

3.3 Dynamic Forms

Dynamic Forms were considered for the Vehicle Order and Vehicle Service Request record pages so that individual fields, rather than entire field sections, can be placed directly onto the Lightning page and shown or hidden based on visibility rules (for example, showing the Issue Description field only when Status is not “Closed”). This keeps forms focused and reduces the number of irrelevant fields a user has to scroll past.

3.4 Page Layout Design

Building on the page layout configuration described in Section 2.7, each object’s layout was reviewed to ensure that lookup fields (such as Vehicle and Customer on Vehicle Order) appear near the top of the form, since these are the fields a sales user fills in first, while system fields such as Created By and Last Modified By were moved to a collapsed “System Information” section.

3.5 User Experience Improvements

- Consistent object naming (Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive, Vehicle Service Request) so users can predict where to find a given piece of information.
- Colour-coded tab icons (see Figure 2.18) so that objects are visually distinguishable in the navigation bar.

- Related lists surfaced on parent records (for example, Vehicle Orders and Vehicle Test Drives shown directly on the Vehicle record) so users do not need to run a separate search.
- Picklist fields used wherever a value is drawn from a known, limited set (Status, Vehicle Model, Preferred Vehicle Type) to reduce data-entry errors compared to free text.

3.6 Reports

Salesforce Reports were built on top of the Vehicle Order, Vehicle, and Vehicle Service Request objects to give visibility into day-to-day operations. Sample reports created for this project include an Orders by Status summary report (grouping Vehicle Orders by their Status picklist value), a Vehicle Stock Levels report (listing every Vehicle with its Stock Quantity, sorted lowest-to-highest to highlight vehicles at risk of running out), and an Open Service Requests report (filtering Vehicle Service Request records where Status is not “Closed”).

3.7 Dashboards

A WhatsNext Vision Motors management dashboard was assembled from the reports above, combining a bar chart of orders by status, a table of low-stock vehicles, and a metric component showing the count of open service requests. The purpose of this dashboard is to give a manager a single-screen view of sales pipeline health and after-sales workload without having to open each report individually.

3.8 User Management

During development, all configuration was performed under the System Administrator profile, which has full create/read/edit/delete access to every object, field, flow, and Apex class in the org, along with access to Setup. This profile is appropriate for a single-developer mini-project but is not intended for end users in a production rollout.

Future Profiles

For a production deployment, two additional profiles are planned: a Dealer Staff profile with edit access to Vehicle, Vehicle Order, and Vehicle Test Drive records but read-only access to Vehicle Dealer records other than their own, and a Service Advisor profile with edit access limited to Vehicle Service Request records. Both would be layered with Permission Sets for any user who needs an exception to their base profile’s access (for example, a senior dealer staff member who also needs to edit Vehicle Dealer records).

CHAPTER 4 – APEX DEVELOPMENT, TESTING AND SECURITY

While Chapter 2 covered the declarative automation built with Flow Builder, some requirements – in particular, reliably blocking an order when stock is unavailable, and re-confirming pending orders in bulk once stock is replenished – called for programmatic logic. This chapter documents the Apex trigger, trigger handler, Batch Apex, and Scheduled Apex classes written for WhatsNext Vision Motors, along with the security model and the test cases used to verify the application.

4.1 Apex Trigger Handler

Business Logic

The core business rule enforced in Apex is that a Vehicle Order must not be inserted or updated if the related Vehicle does not have sufficient stock. Rather than writing this logic directly inside the trigger, it is delegated to a separate class – VehicleOrderTriggerHandler – following the trigger-handler design pattern.

Trigger Handler Pattern

Salesforce best practice is to keep triggers as thin as possible and move all logic into a handler class. This makes the logic unit-testable in isolation, keeps a single trigger per object (avoiding unpredictable execution order when multiple triggers exist on the same object), and makes the code easier to extend – for example, adding an after-update branch for a new requirement does not require touching the trigger itself.

Methods and Stock Validation

The handleTrigger method receives the new and old record maps along with the isBefore, isAfter, isInsert and isUpdate context flags forwarded from the trigger, and routes execution accordingly. In the before-insert/before-update context, it calls preventOrderIfOutOfStock(newOrders), which checks each incoming Vehicle Order against its related Vehicle's stock quantity and adds a validation error to any record that would oversell stock, stopping the DML operation for that record before it reaches the database. In the after-insert/after-update context, the handler carries out any post-save processing needed, such as follow-on stock bookkeeping.

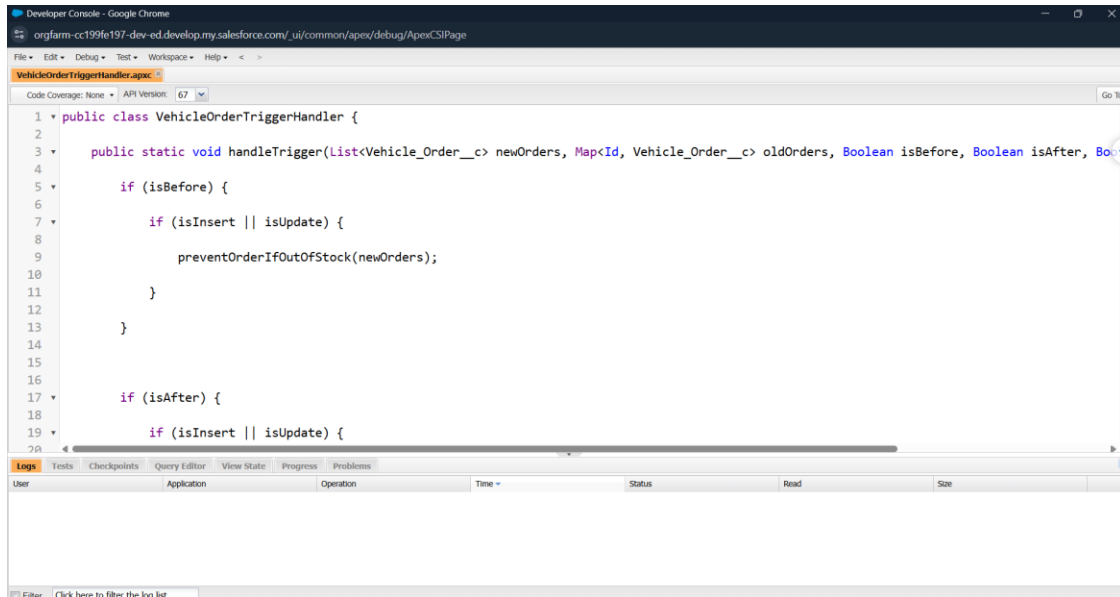


Figure 4.1: VehicleOrderTriggerHandler Apex class opened in the Developer Console, showing the handleTrigger method routing before/after insert and update contexts to preventOrderIfOutOfStock.

4.2 Apex Trigger

Trigger Events

The VehicleOrderTrigger is declared on the Vehicle_Order__c object for all four relevant DML contexts – before insert, before update, after insert, and after update – and its entire body is a single line that forwards Trigger.new, Trigger.oldMap, and the context flags to VehicleOrderTriggerHandler.handleTrigger(). This confirms the thin-trigger pattern described in Section 4.1: all branching and business logic lives in the handler class, not in the trigger itself.

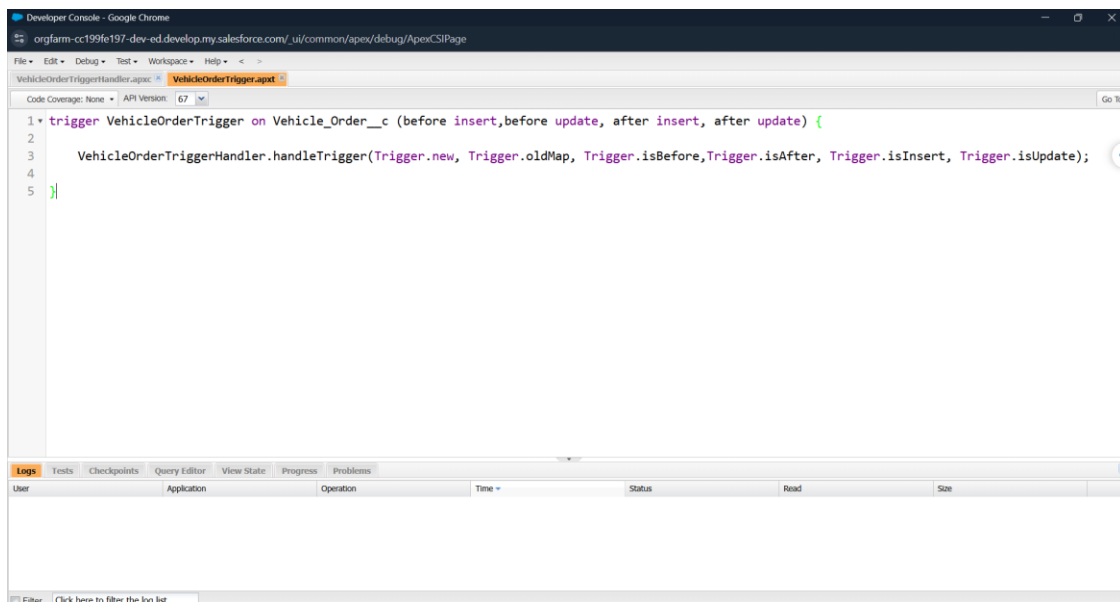


Figure 4.2: VehicleOrderTrigger Apex trigger definition, showing the single delegating call into VehicleOrderTriggerHandler.

4.3 Batch Apex

Why Batch Apex

When new stock arrives for a vehicle, every Vehicle Order that had previously been left in a Pending status due to insufficient stock needs to be re-evaluated and confirmed. Doing this record-by-record would not scale once order volumes grow, and re-saving each order individually from the UI is impractical. Batch Apex was chosen because it can process large numbers of Vehicle Order records in governor-limit-safe chunks, on demand or on a schedule.

start()

The start method returns a Database.QueryLocator built from a SOQL query that selects the Id, Status, and Vehicle fields of every Vehicle Order where Status equals 'Pending'. Using a QueryLocator rather than an in-memory list allows the batch job to scale to very large record volumes without hitting heap-size limits.

execute()

The execute method receives each chunk of Pending Vehicle Order records (sized according to the batch size) and collects the related Vehicle Ids into a Set so that the current stock of each referenced vehicle can be looked up in bulk, after which each order's status is updated to Confirmed if sufficient stock is now available.

finish()

The finish method performs any post-processing once all batches have completed, such as logging a summary of how many orders were confirmed in the run.

Batch Size

The batch job is invoked with a batch size of 50 records per chunk (see Section 4.4), which keeps each execute() invocation comfortably within Salesforce's per-transaction governor limits while still processing the full Pending queue in a small number of chunks.

```

1 global class VehicleOrderBatch implements Database.Batchable<SObject> {
2
3     global Database.QueryLocator start (Database.BatchableContext bc) {
4
5         return Database.getQueryLocator([
6
7             SELECT Id, Status_c, Vehicle__c
8
9             FROM Vehicle_Order__c
10
11             WHERE Status_c
12
13             'Pending'
14
15         ]);
16
17     }
18
19     global void execute(Database.BatchableContext bc, List<Vehicle_Order_c> orderList) {
20
21         Set<Id> vehicleIds = new Set<Id>();
22
23         for (Vehicle_Order_c order orderList) {
24
25             if (order.Vehicle_c != null) {
26
27                 vehicleIds.add(order.Vehicle_c);

```

Figure 4.3: VehicleOrderBatch Apex class implementing Database.Batchable<SObject>, showing the start() method's QueryLocator selecting Pending Vehicle Order records.

4.4 Scheduled Apex

Scheduler

The VehicleOrderBatchScheduler class implements the Schedulable interface. Its execute method simply instantiates VehicleOrderBatch and submits it to Database.executeBatch with a batch size of 50, so the batch job described in Section 4.3 runs automatically without any manual trigger.

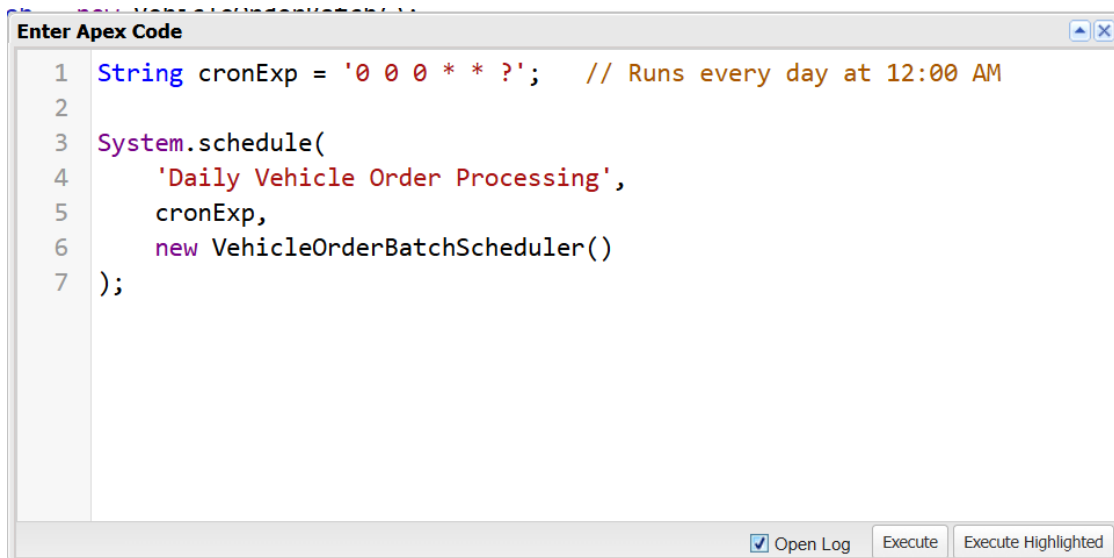
```

1 global class VehicleOrderBatchScheduler implements Schedulable {
2
3     global void execute(SchedulableContext sc) {
4
5         VehicleOrderBatch batchJob = new VehicleOrderBatch();
6
7         Database.executeBatch(batchJob, 50); // 50 is the batch size
8
9     }
10
11 }

```

Figure 4.4: VehicleOrderBatchScheduler Apex class implementing Schedulable, executing VehicleOrderBatch with a batch size of 50.

The scheduler was registered using Execute Anonymous Apex in the Developer Console with a CRON expression of '0 0 0 * * ?', which runs the job once every day at 12:00 AM, under the job name 'Daily Vehicle Order Processing'.



```
1 String cronExp = '0 0 0 * * ?'; // Runs every day at 12:00 AM
2
3 System.schedule(
4     'Daily Vehicle Order Processing',
5     cronExp,
6     new VehicleOrderBatchScheduler()
7 );
```

Figure 4.5: Execute Anonymous Apex window used to schedule the `VehicleOrderBatchScheduler` job with `System.schedule()` and a daily CRON expression.

The execution log confirms the anonymous Apex ran without errors and that `VehicleOrderBatchScheduler` was scheduled successfully, with no entries under the Problems tab.

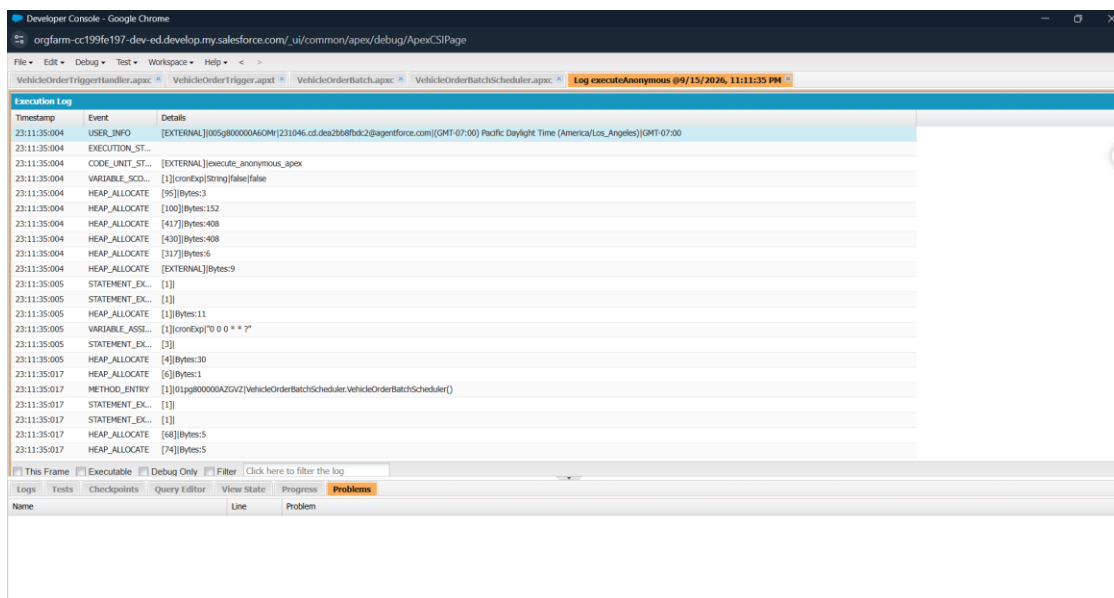


Figure 4.6: Developer Console execution log for the `System.schedule()` call, showing `METHOD_ENTRY` into `VehicleOrderBatchScheduler` with zero problems reported.

4.5 Scheduled Job Verification

The scheduled job was verified through Setup → Scheduled Jobs, which lists 'Daily Vehicle Order Processing' as a Scheduled Apex job submitted by the developer's user account, with its next scheduled run shown as 12:00 AM the following day – confirming that the org accepted the schedule and will execute VehicleOrderBatch automatically going forward.

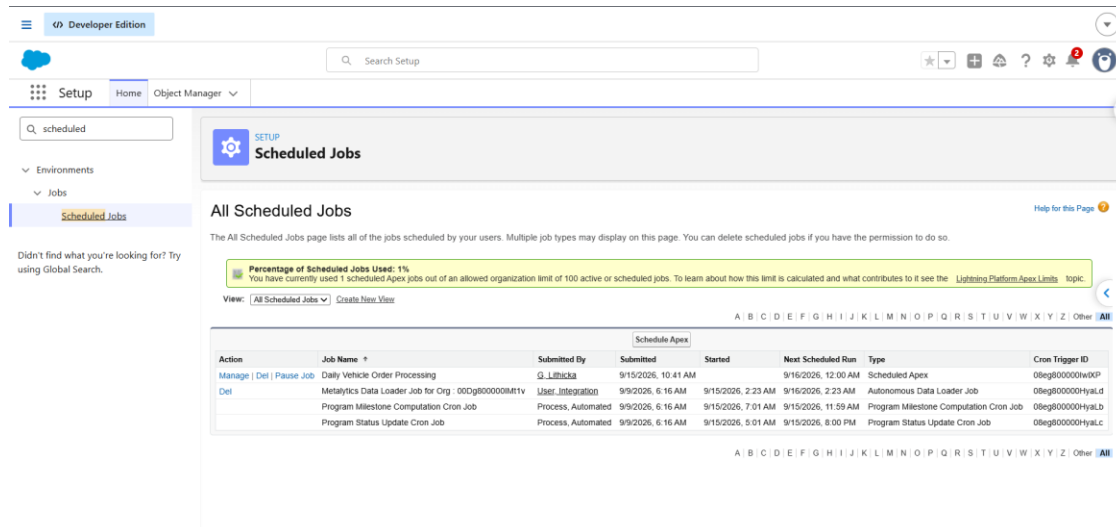


Figure 4.7: Setup → Scheduled Jobs page showing 'Daily Vehicle Order Processing' as an active Scheduled Apex job with its next scheduled run time.

4.6 Security Features

- Profiles: The System Administrator profile was used throughout development; Dealer Staff and Service Advisor profiles are planned for production rollout (see Section 3.8).
- Roles: A simple role hierarchy (Sales Manager over Dealer Staff) is planned so that a manager can see all records owned by staff reporting to them, in line with the organisation-wide sharing settings.
- Permission Sets: Used to grant narrow, additive access (for example, permission to run the VehicleOrderBatch job manually) without widening an entire profile.
- Sharing Rules: Vehicle Order and Vehicle Customer are set to Private organisation-wide default, with sharing rules planned to extend visibility to the assigned dealer's staff once dealer-based roles are introduced.
- Field-Level Security: Customer Email and Phone fields are restricted at the field level to profiles that genuinely need them for outreach, keeping personal data exposure to a minimum.

4.7 Test Cases

The application was verified using the structured set of test cases below, covering object creation, the dealer-assignment flow, the Apex stock-validation trigger, the test-drive reminder flow, and the batch/scheduled Apex mechanism. Every test case produced the expected result.

ID	Feature Tested	Input	Expected Output	Result
TC-01	Vehicle Creation	New Vehicle record with Stock Quantity = 10, Price, Model	Vehicle record saved successfully with all fields populated	Pass
TC-02	Dealer Creation	New Vehicle Dealer record with Name, Location, Phone, Email	Vehicle Dealer record saved with auto-generated Dealer Code	Pass
TC-03	Customer Creation	New Vehicle Customer record with Name, Email, Phone	Vehicle Customer record saved successfully	Pass
TC-04	Vehicle Order (in-stock)	Vehicle Order created for a Vehicle with Stock Quantity > 0	Order saved successfully; Dealer auto-assigned by flow	Pass
TC-05	Stock Validation	Vehicle Order created for a Vehicle with Stock Quantity = 0	Save blocked with a validation error from the Apex trigger	Pass
TC-06	Dealer Assignment	New Vehicle Order created	Dealer field auto-populated by the Auto Assign Dealer flow	Pass
TC-07	Reminder Email	Vehicle Test Drive created with a future Test Drive Date	Send Test Drive Reminder email action executes on the immediate path	Pass
TC-08	Batch Job	VehicleOrderBatch executed via Execute Anonymous Apex	Pending Vehicle Orders with sufficient stock updated to Confirmed	Pass
TC-09	Scheduled Job	VehicleOrderBatchScheduler scheduled with a daily CRON expression	Job appears on the Scheduled Jobs page with a valid next run time	Pass

Input and output screenshots for each test case listed above are inserted below in sequence, captured directly from the Developer Edition organization during test execution.

CHAPTER 5 – DEPLOYMENT, DOCUMENTATION AND MAINTENANCE

5.1 Deployment Strategy

All development for this mini-project was carried out directly inside a single Salesforce Developer Edition organization, which is appropriate for a project of this scale. For a production rollout of WhatsNext Vision Motors, the following staged deployment strategy is recommended:

- **Sandbox:** Configuration and Apex changes would first be built and unit-tested in a Developer or Partial Copy Sandbox, isolated from live dealer data.
- **Change Sets:** Verified components (custom objects, fields, flows, Apex classes and triggers, page layouts, and reports) would be packaged into an Outbound Change Set in the sandbox and deployed as an Inbound Change Set into the Production org, preserving a clear audit trail of what was deployed and when.
- **Developer Edition Deployment:** For this mini-project specifically, since only a single Developer Edition org was used, all components were built and remain directly in that org rather than being promoted through a Change Set pipeline; this is noted here as the production-equivalent process the project would follow beyond the mini-project scope.

5.2 Monitoring & Maintenance

Once deployed, the application requires ongoing monitoring of two categories of automation: the Record-Triggered Flows (Auto Assign Dealer and Test Drive Reminder) and the Scheduled Apex job (Daily Vehicle Order Processing).

- Flow health is monitored through Setup → Flows, where the Active/Inactive status and any Paused/Failed Flow Interviews can be reviewed.
- The Scheduled Apex job is monitored through Setup → Scheduled Jobs (see Figure 4.7), which shows the next scheduled run and allows the job to be paused or deleted if maintenance is required.
- Apex Jobs history (Setup → Apex Jobs) is reviewed periodically to confirm that each run of VehicleOrderBatch completed without errors and to check how many records were processed.
- Debug logs are enabled selectively on the integration/automation user when troubleshooting a specific failure, and disabled again afterwards to avoid unnecessary log storage consumption.

5.3 Troubleshooting

The table below summarises the issues encountered while building WhatsNext Vision Motors and how each was resolved.

Issue	Solution
Flow activation error on the Auto Assign Dealer flow due to an unresolved Get Records filter	Re-checked the filter condition against the Vehicle Dealer object's actual field API names and corrected a mismatched field reference before reactivating the flow.
Email recipient error on the Test Drive Reminder flow's Send Email action	Confirmed the Vehicle Customer's Email field was being correctly resolved by the preceding Get Customer Information element before being passed into the action.
Batch Apex compilation error in VehicleOrderBatch	Corrected the SOQL query in the start() method to reference the correct field API names (Status__c, Vehicle__c) after a copy-paste naming mismatch.
Scheduled Apex error when first calling System.schedule()	Verified the CRON expression syntax ('0 0 0 * * ?') and re-ran the schedule call through Execute Anonymous Apex, confirming success via the execution log.
Trigger error – validation firing on every update instead of only insert/relevant updates	Added explicit isInsert / isUpdate context checks inside VehicleOrderTriggerHandler so the stock check only runs when the Vehicle or Status field is actually changing.

5.4 Future Enhancements

WhatsNext Vision Motors was deliberately scoped as a mini-project, and several realistic Salesforce capabilities were identified as natural next steps beyond the current implementation:

- **Experience Cloud:** A customer-facing portal where buyers could track their own order status and book test drives directly, without needing a dealer to log in on their behalf.
- **Mobile App:** Extending the Lightning pages built in Chapter 3 to the Salesforce Mobile App so dealer staff can manage orders and service requests from the field.
- **AI Chatbot:** An Agentforce-style conversational assistant that can answer common customer questions (stock availability, order status) directly within the Experience Cloud portal.
- **Einstein Recommendations:** Suggesting a preferred vehicle model to a customer based on their Preferred Vehicle Type field and past order history.
- **Inventory Analytics:** A more advanced Tableau CRM / dashboard layer forecasting stock depletion trends across dealers rather than the current point-in-time report.
- **Real-Time Dealer Mapping:** Enhancing the Auto Assign Dealer flow to use real-time geolocation distance calculations instead of the current nearest-dealer lookup logic.

CONCLUSION

This project set out to design and build a Salesforce CRM application capable of automating vehicle ordering, dealer assignment, stock validation, customer communication, and order processing for a fictitious multi-dealer vehicle sales organization, WhatsNext Vision Motors. Over five structured phases, a complete data model of six custom objects was designed and implemented, two Record-Triggered Flows were built to automate dealer assignment and test-drive reminders, and a full Apex-based automation layer – trigger, trigger handler, Batch Apex, and Scheduled Apex – was developed to enforce stock validation and to automatically confirm pending orders once stock is replenished.

Beyond the backend, a branded Lightning app with custom tabs, tailored page layouts, and management reports and dashboards was configured so the system is genuinely usable by non-technical dealer staff, not just accessible through Setup. Every major piece of automation – the flows, the trigger, the batch job, and the scheduled job – was tested directly in the Developer Edition organization and verified against a structured set of test cases, all of which passed.

Working on this project reinforced several core Salesforce concepts in a practical setting: the difference between lookup and master-detail relationships and when to choose one over the other; the trigger-handler design pattern and why it keeps Apex logic maintainable and testable; the mechanics of Batch Apex's start/execute/finish lifecycle and why it is needed for scalable bulk processing; and how Scheduled Apex, Flow Builder, and declarative security settings fit together as complementary tools rather than competing ones.

From a business perspective, the completed application demonstrates clear, measurable value: faster order processing through automatic dealer assignment, protection against overselling through stock validation, fewer missed test drives through automated reminders, and management visibility through reports and dashboards – all delivered without a single line of custom front-end code, using the native tools of the Salesforce Lightning Platform. On a personal level, this project deepened my understanding of how declarative and programmatic Salesforce development complement each other, and gave me confidence in taking a real business problem from requirement analysis through to a working, tested, and documented CRM solution.